MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS – IFNMG
Criado pela Lei no.11.892, de 29/12/2008
Campus Montes Claros

JOÃO KENNEDY SOUZA SOARES
ALEXANDRE JULIO LIMA MENDES

RELATÓRIO : JOB SHOP COM MAKESPAN

Relatório apresentado ao Professor Tadeu Zubaran como parte das exigências de avaliação da disciplina de Algoritmos para Grafos do Curso de Bacharelado em Ciência da Computação do Instituto Federal do Norte de Minas Gerais, Campus Montes Claros.

MONTES CLAROS - MG

2026

Relatório de Desenvolvimento: Minimização de Makespan no Job Shop Scheduling

1. Introdução

Este relatório detalha o projeto e a implementação de uma solução na linguagem C++ para o Problema de Job Shop Scheduling (JSSP), amplamente utilizado em ambientes industriais, onde temos um conjunto de N trabalhos (Jobs) que precisam ser processados por um conjunto de M máquinas. Dessa forma, a complexidade do problema reside em duas restrições fundamentais. A primeira estabelece que cada Job possui uma rota específica e obrigatória de máquinas pela qual deve passar, como exemplo, um trabalho passa pela máquina A e somente depois pela B. Segundamente é a restrição de capacidade, que determina que uma máquina só pode processar uma única operação por vez, de forma contínua e sem interrupções.

2. Visão Geral e Objetivos do Projeto

Nesse cenário restrito, o objetivo principal do algoritmo desenvolvido é a minimização do Makespan ($C_{\max}$). O Makespan é definido como o tempo de término do último Job a sair da fábrica. Minimizar esse valor significa encontrar a ordem perfeita de execução das tarefas nas máquinas, reduzindo assim o tempo ocioso, eliminando gargalos e fazendo com que todo o conjunto de peças seja concluído no menor tempo global possível. Sendo assim, o desenvolvimento desta solução foi pautado na modelagem matemática do problema utilizando Grafos Disjuntivos, culminando na aplicação de uma Busca Local.

3. Desenvolvimento e Tecnologias

Para a construção do projeto, as seguintes ferramentas e bibliotecas integradas foram utilizadas:

- **Linguagem:** C++, compilado via Makefile.
- **Organização de Código:**
  - **JobShop.h:** Estruturas de dados (GrafoDisjuntivo) e declarações das funções.
  - **JobShop.cpp:** Implementação dos algoritmos como (Parsing,

Algoritmo de Kahn, Programação Dinâmica, Hill Climbing).

    - **main.cpp:** Ponto de entrada, leitura dos argumentos (.psi), cronômetro de tempo com a biblioteca "chrono" e exibe os resultado obtidos.

- **Instâncias:** O programa foi testado em benchmarks disponibilizados com as instâncias de Taillard "*ta01* a *ta80*", fazendo o parsing do cabeçalho e matriz de Costs seguidos dos subgrafos das operações.

4. Modelagem do Problema

O problema foi modelado utilizando o modelo matemático de um Grafo Disjuntivo, o qual nesse modelo os vértices representam as operações individuais, que consistem em uma etapa específica de um Job sendo processada em uma determinada máquina. As ligações do grafo se dividem em dois tipos. As arestas conjuntivas (direcionadas) ditam a precedência, indicando a ordem obrigatória das máquinas para um mesmo Job. Já as arestas disjuntivas (inicialmente não-direcionadas) representam a restrição de capacidade das máquinas, as quais garantem que duas operações não ocupem o mesmo equipamento simultaneamente. Dessa forma, o papel central do algoritmo de escalonamento é encontrar a melhor orientação para essas arestas disjuntivas de forma a minimizar o tempo total sem gerar ciclos.

As decisões de projeto, propomos por realizar uma linearização de índices em vez de utilizar matrizes bidimensionais. Os vértices e suas conexões foram mapeados para Arrays como T (tempos), M (máquinas), J (jobs), e seus sucessores e antecessores (*SucJ, AntJ, SucM, AntM*). Para que isso fosse possível, cada par referente a um Job e a uma Máquina foi convertido para um identificador único através da fórmula matemática de linearização: ($ID = j * total_maquinas + m$).

A principal motivação técnica para o uso de índices linearizados e vetores paralelos é a maximização da localidade de referência em cache do processador. Sabendo que algoritmos de grafos realizam saltos constantes e acessos aleatórios na memória, a manutenção dos dados em blocos contíguos reduz as falhas de cache e elimina a sobrecarga de desreferenciamento de ponteiros característico das estruturas puramente orientadas a objetos. Consequentemente, essa decisão arquitetural garante um tempo de execução de baixíssima latência, algo estritamente necessário para viabilizar as iterações exigidas pela busca local.

5. Heurística Construtiva (Solução Inicial)

Para resolver o problema das arestas disjuntivas e gerar um agendamento viável inicial, implementou-se um algoritmo construtivo guloso, na qual o seu funcionamento baseia-se na utilização de uma fila do tipo FIFO que gerencia as operações disponíveis. Apenas as tarefas que não possuem predecessores pendentes no seu respectivo Job têm permissão para entrar nessa fila. Ao retirar uma operação da fila, ela é imediatamente agendada na sua máquina correspondente, o que cria as arestas de precedência na máquina (*SucM* e *AntM*) conectando-a à operação anterior daquele equipamento. Feito isso, o sucessor lógico dessa operação no seu Job de origem é destrancado e passa a compor a fila.

A escolha por esse critério guloso pautado na "ordem de chegada" é fundamental porque garante uma alocação rápida, com complexidade $O(N)$, e produz um agendamento livre de ciclos estruturais conhecidos como Deadlocks. Como o avanço na linha do tempo é sempre progressivo e ditado pela disponibilidade real das peças, impossibilita-se a geração de cenários onde uma máquina A espera infinitamente pela máquina B enquanto a máquina B espera pela máquina A.

6. Caminhada Topológica e Cálculo do Makespan

Com todas as arestas disjuntivas devidamente orientadas pela heurística, o tempo total de término, ou Makespan, precisa ser avaliado. Para isso, aplicamos a Caminhada Topológica utilizando o algoritmo de Kahn, baseado no cálculo iterativo de graus de entrada. Além de fornecer uma sequência de processamento cronológica segura de todas tarefas, a ordenação topológica atua como um mecanismo de validação: se, ao final do processo, o número de vértices iterados for menor que o número total de operações, constata-se a formação de um ciclo, invalidando a solução gerada.

Tendo a garantia de um grafo acíclico e a sua respectiva ordem topológica, o algoritmo utiliza a técnica de Programação Dinâmica para o cálculo do caminho máximo. O tempo de início de cada tarefa é atualizado progressivamente, assumindo sempre o valor máximo entre os tempos de término de todos os seus antecessores diretos. O maior registrado ao concluir a avaliação global define o Makespan. Por fim, reconstruindo o caminho a partir desse último nó e retrocedendo através dos marcadores de "pais", o algoritmo revela o Caminho Crítico, e o seu gargalo da linha de produção.

7. Busca Local e Vizinhança de Blocos

Embora a solução gerada pela heurística inicial seja viável, ela raramente apresenta um tempo ótimo de produção. Visando refinar o Makespan, foi implementada uma heurística de Hill Climbing fortemente ancorada no teorema da Vizinhança de Blocos (Vizinhança N3), consolidada com base nos trabalhos de teóricos como Dell'Amico e Trubian (1993). A fundamentação teórica dita que inverter arestas disjuntivas aleatoriamente resultaria em um espaço de busca fatorial inviável O(n!). Contudo, a literatura de Scheduling comprova matematicamente que alterar a ordem de operações que possuem folga temporal não produz nenhum efeito no tempo final. Além disso, permutar operações que estão situadas no meio de um bloco de tarefas contíguas na mesma máquina pertencentes ao Caminho Crítico jamais reduz o Makespan de forma imediata.

Sendo assim, o algoritmo foi projetado para extrair o Caminho Crítico da solução atual e identificar sistematicamente os Blocos de Máquina, que são agrupamentos de operações adjacentes no caminho crítico processadas no mesmo equipamento. Como decisão de otimização estrita, o código restringe as inversões de ordem (swaps) exclusivamente às extremidades desses blocos, focando apenas na permutação do primeiro e do último par de tarefas contíguas de cada bloco identificado.

Essa abordagem reduz drasticamente o fator de ramificação da busca local, conferindo altíssima eficiência ao processo de refinamento. O sistema testa apenas as permutações com real potencial de redução do tempo do gargalo. A cada alteração realizada nos ponteiros das máquinas (SucM e AntM), a ausência de ciclos é prontamente validada pela Caminhada Topológica. Se o novo agendamento for factível e apresentar um Makespan inferior ao atual, a mudança é aceita permanentemente. Caso contrário, se a solução piorar ou gerar um ciclo inválido, o estado do grafo sofre um processo imediato de Rollback, restaurando as conexões da máquina para o formato da iteração anterior.

8. Resultados e Conclusões

O desempenho do algoritmo foi avaliado comparando-se o Makespan Obtido (Cmax) com a Best Known Solution (BKS - Melhor Solução Conhecida) através do cálculo do Gap percentual.

Instância	Makespan Obtido	Best Known Solution	Gap	Gap (%)
ta01Jsp.psi	1503	1231	272	22.10%
ta02Jsp.psi	1481	1244	237	19.05%
ta03Jsp.psi	1478	1218	260	21.35%
ta04Jsp.psi	1548	1175	373	31.74%
ta05Jsp.psi	1625	1224	401	32.76%
ta06Jsp.psi	1492	1238	254	20.52%
ta07Jsp.psi	1447	1227	220	17.93%
ta08Jsp.psi	1397	1217	180	14.79%
ta09Jsp.psi	1579	1274	305	23.94%
ta10Jsp.psi	1528	1241	287	23.13%
ta11Jsp.psi	1840	1357	483	35.59%
ta12Jsp.psi	1790	1367	423	30.94%
ta13Jsp.psi	1754	1342	412	30.70%
ta14Jsp.psi	1638	1345	293	21.78%
ta15Jsp.psi	1740	1339	401	29.95%
ta16Jsp.psi	1658	1360	298	21.91%
ta17Jsp.psi	1997	1462	535	36.59%
ta18Jsp.psi	1854	1396	458	32.81%
ta19Jsp.psi	1742	1332	410	30.78%
ta20Jsp.psi	1693	1348	345	25.59%
ta21Jsp.psi	1999	1642	357	21.74%
ta22Jsp.psi	2129	1600	529	33.06%
ta23Jsp.psi	1929	1557	372	23.89%
ta24Jsp.psi	2017	1644	373	22.69%
ta25Jsp.psi	2045	1595	450	28.21%
ta26Jsp.psi	2053	1643	410	24.95%
ta27Jsp.psi	2224	1680	544	32.38%
ta28Jsp.psi	2130	1603	527	32.88%
ta29Jsp.psi	1897	1625	272	16.74%
ta30Jsp.psi	1966	1584	382	24.12%
ta31Jsp.psi	2221	1764	457	25.91%
ta32Jsp.psi	2367	1784	583	32.68%
ta33Jsp.psi	2353	1791	562	31.38%
ta34Jsp.psi	2320	1829	491	26.85%

ta35Jsp.psi	2407	2007	400	19.93%
ta36Jsp.psi	2379	1819	560	30.79%
ta37Jsp.psi	2289	1771	518	29.25%
ta38Jsp.psi	2133	1673	460	27.50%
ta39Jsp.psi	2530	1795	735	40.95%
ta40Jsp.psi	2272	1669	603	36.13%
ta41Jsp.psi	2643	2005	638	31.82%
ta42Jsp.psi	2761	1937	824	42.54%
ta43Jsp.psi	2666	1846	820	44.42%
ta44Jsp.psi	2772	1979	793	40.07%
ta45Jsp.psi	2704	2000	704	35.20%
ta46Jsp.psi	2755	2004	751	37.48%
ta47Jsp.psi	2566	1889	677	35.84%
ta48Jsp.psi	2573	1937	636	32.83%
ta49Jsp.psi	2723	1961	762	38.86%
ta50Jsp.psi	2627	1923	704	36.61%
ta51Jsp.psi	3669	2760	909	32.93%
ta52Jsp.psi	3527	2756	771	27.98%
ta53Jsp.psi	3454	2717	737	27.13%
ta54Jsp.psi	3300	2839	461	16.24%
ta55Jsp.psi	3544	2679	865	32.29%
ta56Jsp.psi	3643	2781	862	31.00%
ta57Jsp.psi	3456	2943	513	17.43%
ta58Jsp.psi	3488	2885	603	20.90%
ta59Jsp.psi	3465	2655	810	30.51%
ta60Jsp.psi	3140	2723	417	15.31%
ta61Jsp.psi	3730	2868	862	30.06%
ta62Jsp.psi	3959	2869	1090	37.99%
ta63Jsp.psi	3595	2755	840	30.49%
ta64Jsp.psi	3301	2702	599	22.17%
ta65Jsp.psi	3503	2725	778	28.55%
ta66Jsp.psi	3798	2845	953	33.50%
ta67Jsp.psi	3916	2825	1091	38.62%
ta68Jsp.psi	3567	2784	783	28.12%
ta69Jsp.psi	3791	3071	720	23.45%

ta70Jsp.psi	4047	2995	1052	35.13%
ta71Jsp.psi	6573	5464	1109	20.30%
ta72Jsp.psi	6151	5181	970	18.72%
ta73Jsp.psi	6781	5568	1213	21.79%
ta74Jsp.psi	6093	5339	754	14.12%
ta75Jsp.psi	6682	5392	1290	23.92%
ta76Jsp.psi	6342	5342	1000	18.72%
ta77Jsp.psi	6440	5436	1004	18.47%
ta78Jsp.psi	6360	5394	966	17.91%
ta79Jsp.psi	6138	5358	780	14.56%
ta80Jsp.psi	6194	5183	1011	19.51%

Ao analisar os resultados das instâncias de Taillard, foi observado:

- Uma variação alta entre instâncias, variando entre 14,12% (ta74) e 44,42% (ta43).
- As instâncias de maior escala (ta71 a ta80) apresentaram resultados inferiores a 24%, destacando-se o ta74 (14,12%) e ta79 (14,56%). Isso sugere que a Busca Local utilizada mantém uma estabilidade relativa mesmo com o aumento do espaço de busca.
- O grupo de instâncias entre ta39 e ta50 apresentou os maiores intervalos (frequentemente acima de 35%).

A escolha pela linearização de índices e o foco no caminho crítico permitiu que o algoritmo processasse as instâncias de forma extremamente rápida, confirmando que priorizar as permutações que efetivamente impactam o gargalo produtivo é uma estratégia eficaz para reduzir drasticamente o tempo de processamento.

9. Referências

GONÇALVES, José Fernando; RESENDE, Mauricio G. C. An extended Akers graphical method with a biased random-key genetic algorithm for job-shop scheduling. **International Transactions in Operational Research**, v. 21, n. 2, p. 215-246, 2014.

JAIN, A. S.; MEERAN, S. Deterministic job-shop scheduling: Past, present and future. **European Journal of Operational Research**, v. 113, n. 2, p. 390-434, 1999.

DELL'AMICO, Mauro; TRUBIAN, Marco. Applying tabu search to the job-shop scheduling problem. **Annals of Operations Research**, v. 41, n. 3, p. 231-252, 1993.

JSPLIB. **Job Shop Scheduling Library.** Disponível em: https://scheduleopt.github.io/benchmarks/jsplib/. Acesso em: 24 de abril de 2026.

WEISE, Thomas. **JSSP Instances and Results.** Repositório GitHub, 2024. Disponível em: https://github.com/thomasWeise/jsspInstancesAndResults. Acesso em: 24 de abril de 2026.